

CHAPITRE 4 : ARCHITECTURE ET CHOIX TECHNIQUES

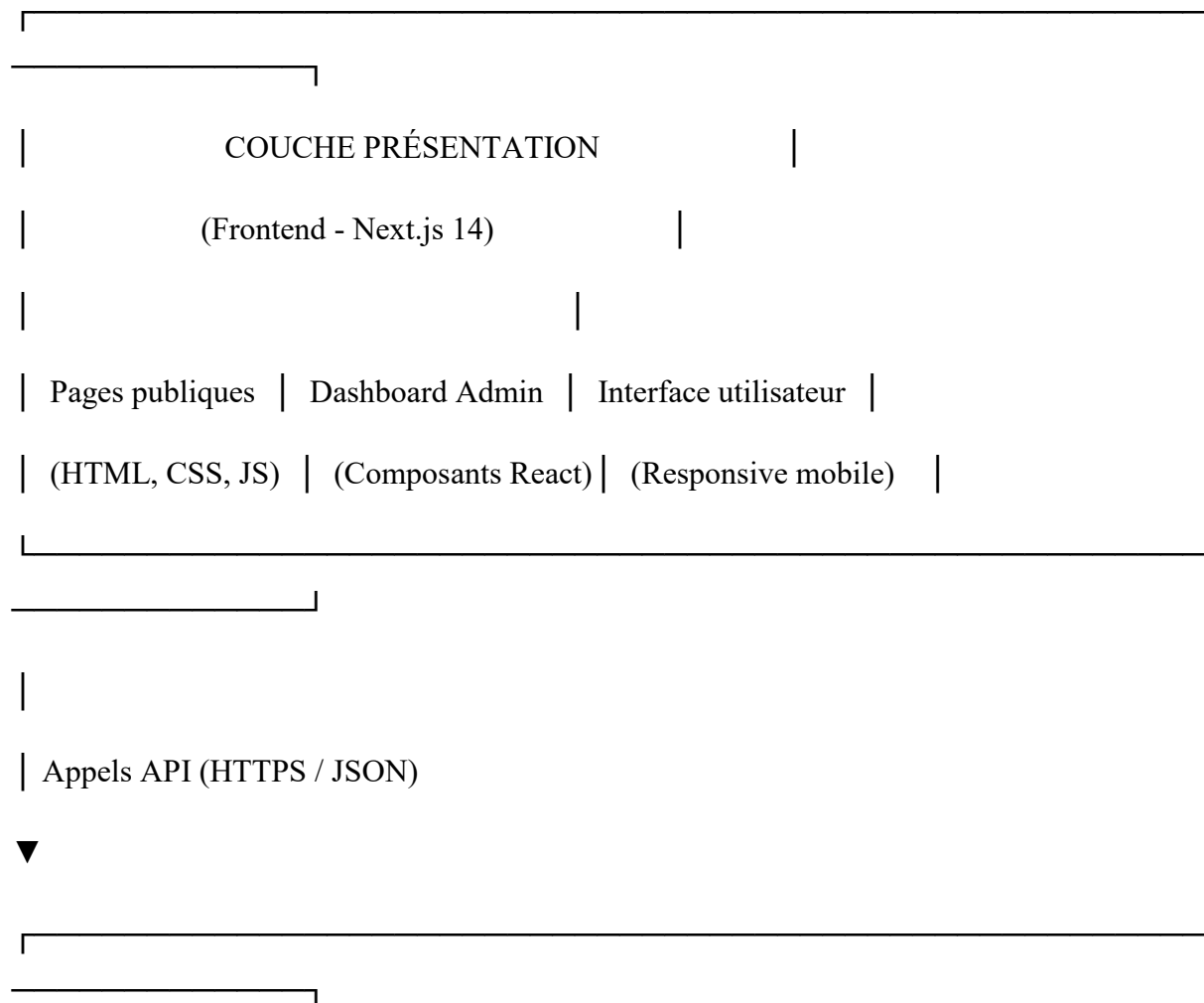
4.1 Architecture Globale du Système

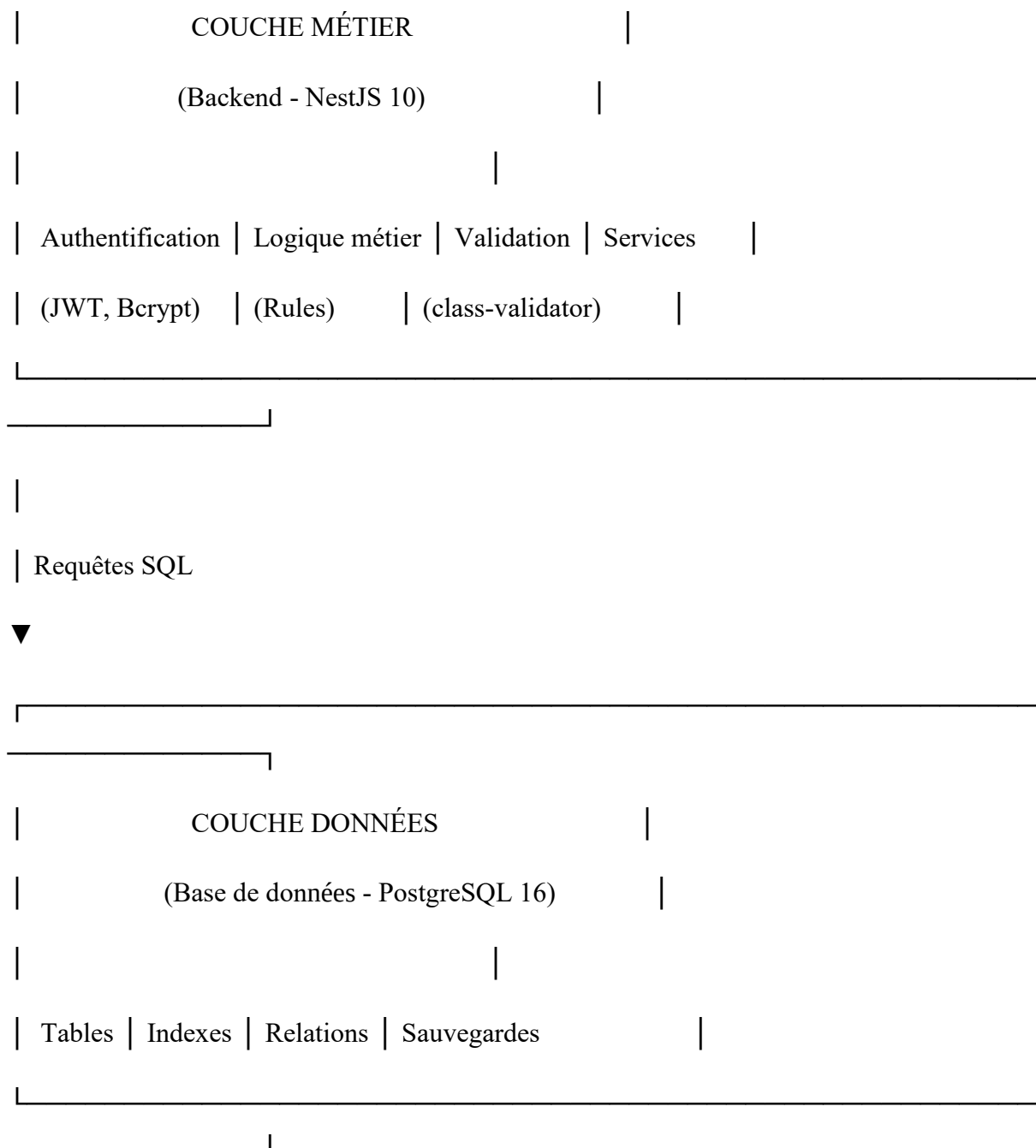
4.1.1 Présentation de l'Architecture

L'architecture du site Y-MAD suit le modèle d'architecture 3-tiers (trois couches) qui sépare clairement les différentes responsabilités du système. Cette séparation rend l'application plus facile à maintenir, à faire évoluer et à tester.

Les trois couches de l'architecture :

text





4.1.2 Schéma d'Architecture Détaillé

Voici comment les différents composants communiquent entre eux :

text

UTILISATEUR (Navigateur)

|

| 1. Visite y-mad.mg



| _____ |

| Cloudflare | ← Protection DDoS + Cache + HTTPS

| (CDN + SSL) |

| _____ |

| 2. Requête HTTPS



| _____ |

| VPS Contabo |

| (Ubuntu 24.04) |

| _____ |

| _____ |

| | Nginx | | ← Reverse proxy + Gestion SSL

| | (Port 80/443) |

| _____ |

| | _____ |

| _____▼_____ |

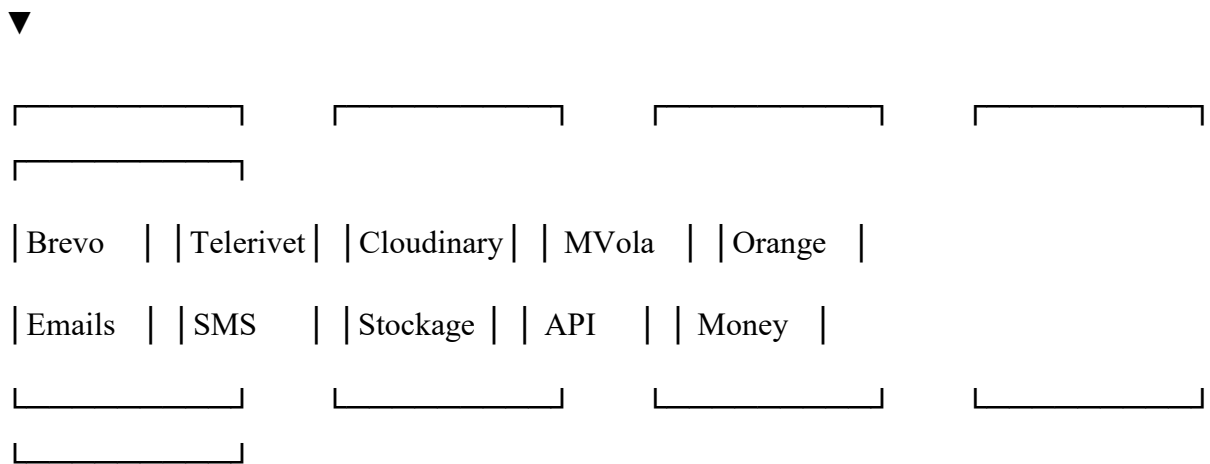
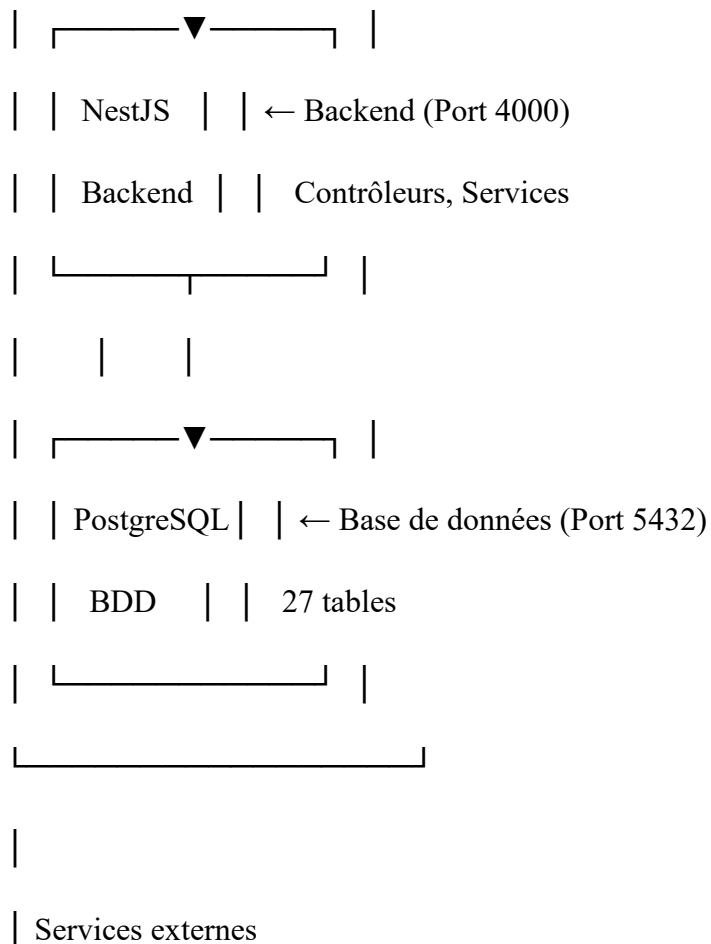
| | Next.js | | ← Frontend (Port 3000)

| | Frontend | | Pages, composants React

| _____ |

| | _____ |

| | API |



4.1.3 Flux de Données - Cas Concret "Candidature Emploi"

Prenons l'exemple d'un jeune qui postule à une offre d'emploi pour comprendre le flux complet des données :

text

ÉTAPE 1 : Jeune visite la page d'une offre d'emploi

Jeune → Navigateur → /jobs/123 → Next.js → API → PostgreSQL

Résultat : Affiche la description de l'offre

ÉTAPE 2 : Jeune remplit le formulaire et envoie sa candidature

Jeune → Formulaire avec CV PDF et photo

↓

Next.js (Frontend) : Validation des champs + taille fichiers (<5Mo)

↓

Appel API POST /jobs/apply (avec fichiers)

↓

NestJS (Backend) :

- Vérifie que l'offre existe et est publiée
- Vérifie qu'il n'y a pas déjà une candidature (email + job_offer_id)
- Upload des fichiers vers Cloudinary
- Sauvegarde en base de données
- Envoi email de confirmation via Brevo

↓

Retour au Frontend : "Candidature envoyée avec succès"

↓

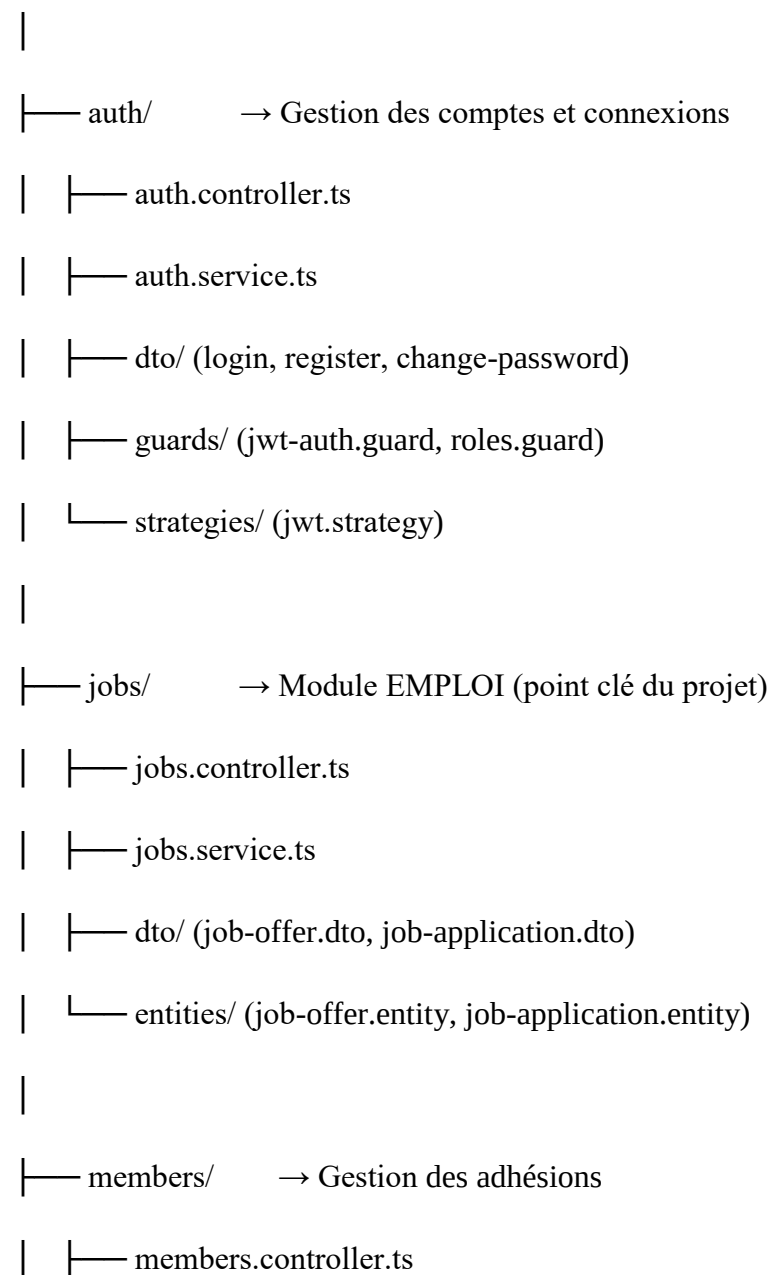
Jeune : Reçoit email de confirmation

4.1.4 Organisation Modulaire du Backend

L'architecture du backend NestJS est organisée en modules indépendants. Chaque module regroupe les fichiers liés à une fonctionnalité spécifique.

text

backend/src/modules/



- | |— members.service.ts
- | |— pdf.service.ts (génération cartes)
- | |— qrcode.service.ts
- |
- |— events/ → Gestion des événements
- |— projects/ → Gestion des projets
- |— donations/ → Gestion des dons (MVola, Orange Money)
- |— blog/ → Articles bilingues
- |— payments/ → Intégration paiements Mobile Money
- |— upload/ → Gestion des fichiers (Cloudinary)

4.1.5 Communication Frontend/Backend

La communication entre le frontend Next.js et le backend NestJS se fait via des appels API REST. Voici les principaux points d'accès :

Méthode HTTP	URL	Rôle	Authentification
POST	/auth/register	Créer un compte	Non
POST	/auth/login	Se connecter	Non
GET	/auth/profile	Voir son profil	Oui (JWT)
GET	/jobs/offers	Lister les offres	Non
GET	/jobs/offers/:id	Détail d'une offre	Non
POST	/jobs/apply	Postuler à une offre	Non (mais lié si connecté)
GET	/jobs/applications/my	Mes candidatures	Oui
GET	/jobs/offers/:id/applications	Voir candidatures d'une offre	Oui (admin)

POST	/members	Devenir membre	Oui
POST	/donations	Faire un don	Non
POST	/events/register	S'inscrire à un événement	Oui
GET	/projects	Voir les projets	Non
GET	/blog	Voir les articles	Non

4.1.6 Sécurité des Échanges

Tous les échanges entre le navigateur et le serveur sont protégés :

text

1. HTTPS (TLS 1.3)

Toutes les données sont chiffrées. Un attaquant qui intercepte la communication ne peut pas lire les informations (email, mot de passe, CV, etc.)

2. Tokens JWT

Après connexion, le serveur génère un token signé. Le client l'envoie dans chaque requête (header Authorization: Bearer token). Le token expire après 7 jours.

3. Rate Limiting

Limite à 100 requêtes par minute par adresse IP. Empêche les attaques par force brute.

4. Validation des entrées

Toutes les données envoyées par le client sont validées avant traitement (format email, taille fichiers, types autorisés).

4.2 Choix Technologiques

4.2.1 Stack Technologique Complet

Le tableau ci-dessous présente l'ensemble des technologies choisies pour le projet Y-MAD :

Couche	Technologie	Version	Rôle
Frontend	Next.js	14	Framework React pour le site public et dashboard
TypeScript	5.x		Langage typé pour du code fiable
Tailwind CSS	3.x		Framework CSS pour design responsive
React Query	5.x		Gestion des appels API et cache
Axios	1.x		Client HTTP pour appels API
Backend	NestJS	10.x	Framework Node.js structuré en modules
TypeScript	5.x		Langage typé
JWT	9.x		Authentification par tokens
Bcrypt	5.x		Hachage des mots de passe
TypeORM	0.3.x		ORM pour PostgreSQL
Base de données	PostgreSQL	16	Base de données relationnelle
Infrastructure	Docker	24.x	Conteneurisation
Nginx	1.24		Reverse proxy
Cloudflare	-		CDN + SSL + Protection DDoS
Services externes	Cloudinary	-	Stockage CV, photos, documents
Brevo (Sendinblue)	-		Envoi emails transactionnels
Telerivet	-		Envoi SMS Madagascar
MVola API	-		Paiement mobile (Telma)
Orange Money API	-		Paiement mobile (Orange)

Déploiement GitHub Actions - CI/CD automatique

VPS Contabo - Hébergement (4 vCPU, 6 Go RAM)

4.2.2 Justification des Choix pour le Frontend

Pourquoi Next.js plutôt que React seul ?

React seul crée une Single Page Application (SPA) où tout le contenu est chargé en JavaScript. Cela pose deux problèmes pour Y-MAD :

Problème	Explication	Solution avec Next.js
----------	-------------	-----------------------

Référencement SEO	Google doit exécuter JavaScript pour voir le contenu. Certains moteurs ne le font pas bien.	Next.js génère le HTML sur le serveur. Le contenu est visible immédiatement par les moteurs de recherche.
-------------------	---	---

Premier chargement	L'utilisateur attend que tout le JS soit téléchargé (plusieurs secondes sur 4G à Madagascar)	Le HTML arrive directement, l'utilisateur voit la page plus vite
--------------------	--	--

Pourquoi Tailwind CSS ?

Tailwind CSS est un framework "utility-first". Au lieu d'écrire du CSS personnalisé, on utilise des classes prédéfinies.

jsx

// Sans Tailwind : il faut écrire du CSS séparé

```
<div className="card">...</div>
```

// style.css : .card { padding: 1rem; border-radius: 0.5rem; }

// Avec Tailwind : tout est dans le JSX

```
<div className="p-4 rounded-lg">...</div>
```

Avantages pour Y-MAD :

Développement plus rapide : pas besoin d'aller dans des fichiers CSS séparés

Responsive intégré : classes comme md:w-1/2 pour différentes tailles d'écran

85% des Malgaches utilisent leur smartphone → le responsive est critique

Taille finale réduite grâce à l'élimination des classes non utilisées

4.2.3 Justification des Choix pour le Backend

Pourquoi NestJS plutôt qu'Express classique ?

Critère	Express.js	NestJS
---------	------------	--------

Structure	Libre, chacun fait à sa manière	Organisation standardisée (modules, contrôleurs, services)
-----------	---------------------------------	--

TypeScript	À configurer manuellement	Intégré nativement
------------	---------------------------	--------------------

Maintenabilité	Difficile sur gros projet	Facile grâce à la séparation en modules
----------------	---------------------------	---

Documentation	APIs externes Swagger intégré automatiquement
---------------	---

Exemple concret : Avec NestJS, un module "emploi" regroupe tout ce qui concerne les offres et candidatures. Un nouveau développeur comprend immédiatement où chercher.

Pourquoi PostgreSQL comme base de données ?

Critère PostgreSQL MySQL SQLite

JSON/JSONB ☐ Très bon support ☐ Support limité ☐ Non

Concurrence ☐ Excellente ☐ Bonne ☐ Impossible (fichier unique)

Type UUID ☐ Natif ☐ Non standard ☐ Non

Performances ☐ Très bonnes ☐ Bonnes ☐ Faibles (multi-utilisateurs)

Raison spécifique pour Y-MAD : Le stockage des données JSON (before_ymad et after_ymad) est essentiel pour mesurer l'impact social. PostgreSQL gère cela parfaitement.

4.2.4 Alternatives Écartées et Justification

Technologie Alternatives écartées Raison de l'abandon

Base de données MySQL PostgreSQL gère mieux les JSON et les types UUID

Frontend Vue.js, Angular Next.js a le meilleur SEO et le plus grand écosystème
React

Backend Express.js, Django (Python) NestJS a une architecture plus propre et
TypeScript natif

CSS Styled-components, Sass Tailwind est plus rapide à développer et responsive par
défaut

ORM Prisma TypeORM est mieux intégré avec NestJS

Paiement Stripe, PayPal MVola et Orange Money sont les seuls utilisés localement à
Madagascar

Email Mailgun, SendGrid Brevo a le plan gratuit le plus généreux (1000 emails/jour)

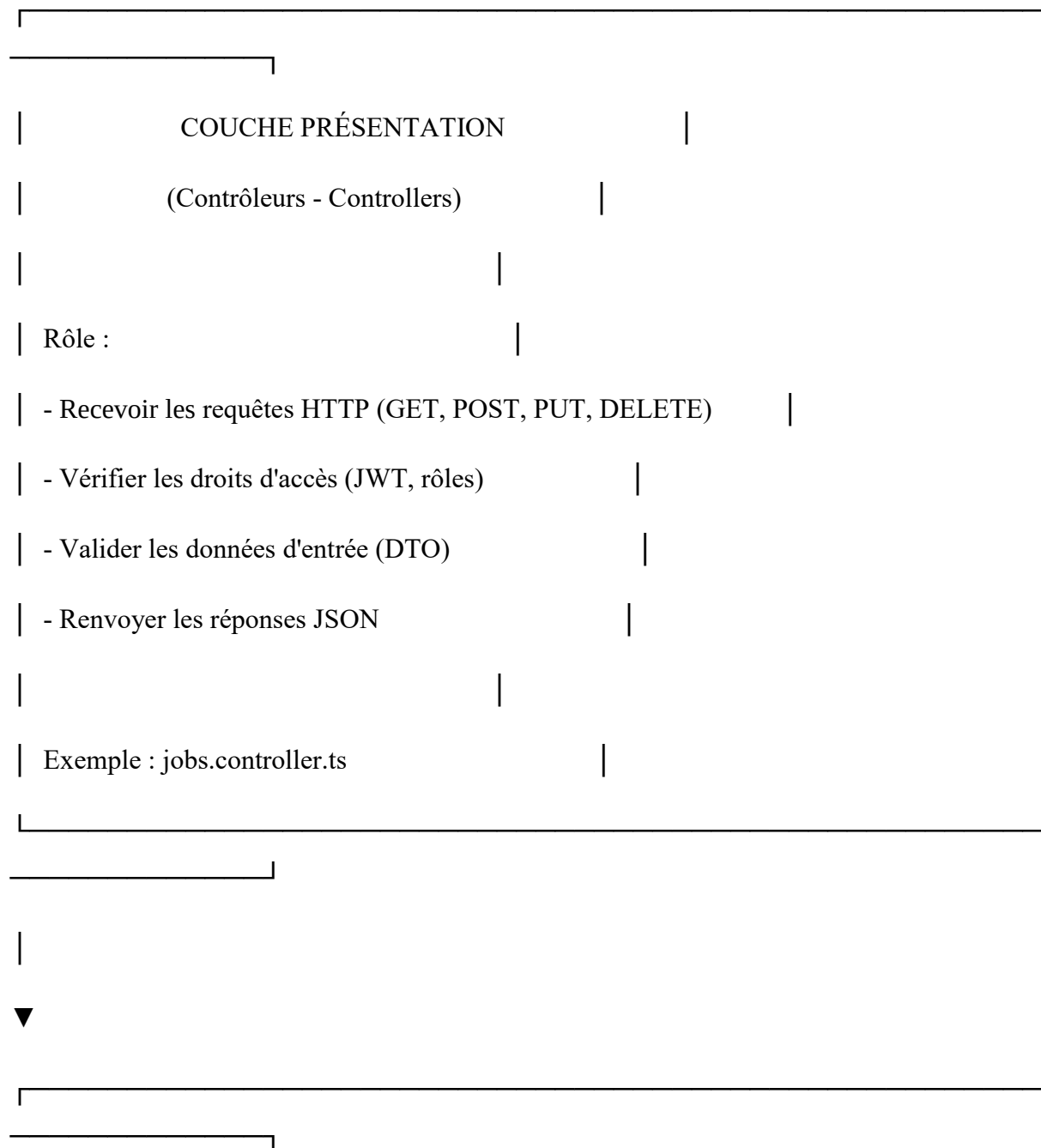
Stockage AWS S3 Cloudinary est gratuit jusqu'à 25 Go et optimise
automatiquement les images

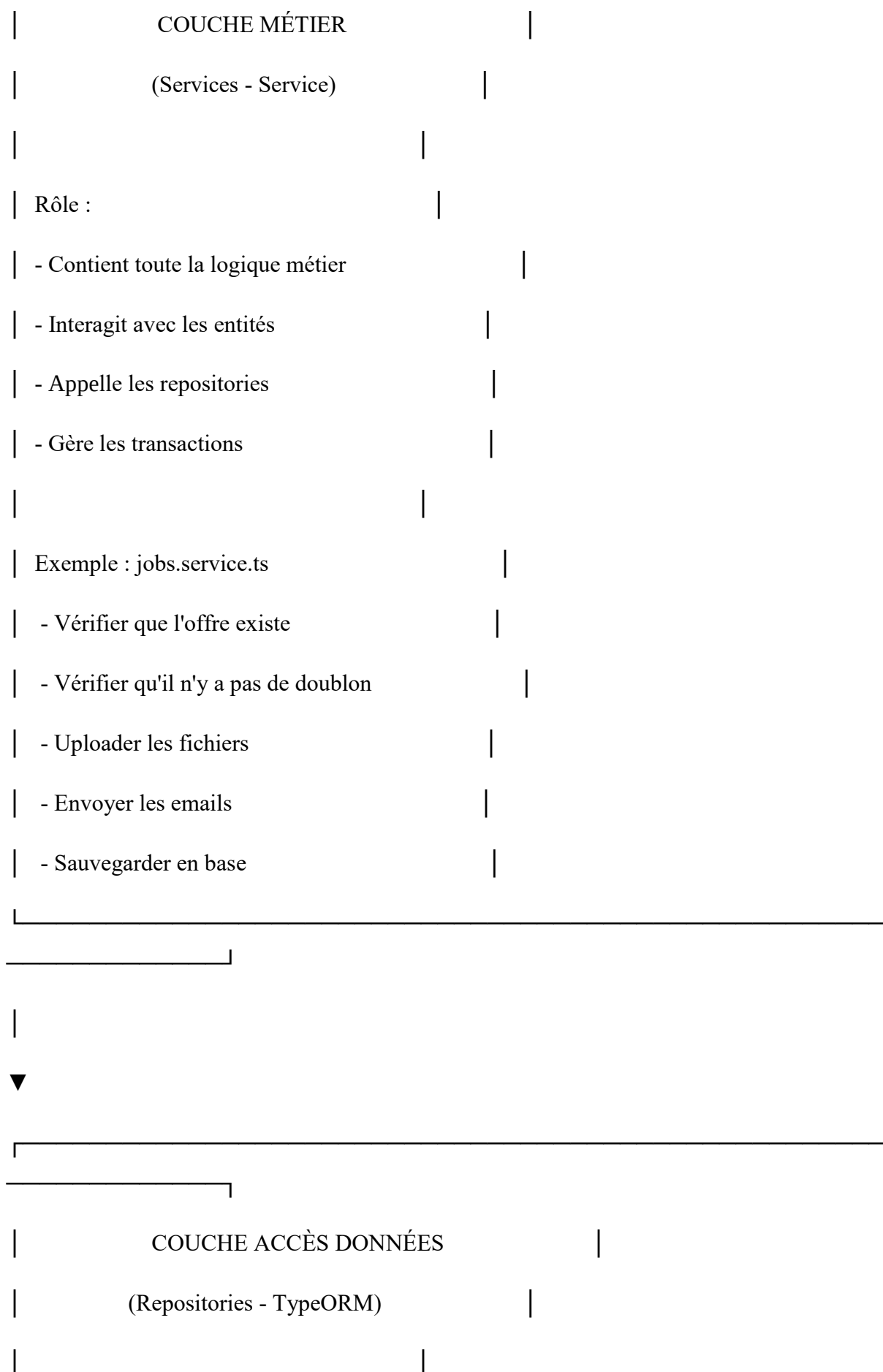
4.3 Architecture Backend et Sécurité

4.3.1 Organisation en Couches (Architecture MVC)

Le backend NestJS suit l'architecture MVC (Modèle - Vue - Contrôleur), adaptée pour une API REST. La "Vue" est remplacée par les réponses JSON.

text





Rôle :	
- Exécuter les requêtes SQL	
- Gérer les relations entre tables	
- Appliquer les filtres et tris	
Exemple : <code>this.jobOfferRepository.save(application)</code>	
▼	
BASE DE DONNÉES	
(PostgreSQL)	

4.3.2 Exemple Concret : Création d'une Candidature

Voici le parcours d'une candidature à travers les différentes couches :

1. Contrôleur (jobs.controller.ts)

typescript

```
@Post('apply')
```

```
@Public() // Accessible sans connexion
```

```
async apply(@Body() applyDto: ApplyToJobDto) {  
  
  return this.jobsService.applyToJob(applyDto);  
  
}
```

2. Service (jobs.service.ts) - Logique métier

typescript

```
async applyToJob(applyDto: ApplyToJobDto) {  
  
  // Vérifier que l'offre existe et est publiée  
  
  const offer = await this.jobOfferRepository.findOne({  
  
    where: { id: applyDto.jobOfferId, status: 'published' }  
  
  });  
  
  
  
  // Vérifier qu'il n'y a pas de doublon (même email, même offre)  
  
  const existing = await this.jobApplicationRepository.findOne({  
  
    where: {  
  
      jobOfferId: applyDto.jobOfferId,  
  
      email: applyDto.email  
  
    }  
  
  });  
  
  
  
  // Traiter l'upload des fichiers  
  
  const cvUrl = await this.uploadService.uploadFile(applyDto.cv);
```



```
// Envoyer email de confirmation
```

// \$2b\$10\$N9qo8uLOickgx2ZMRZoMy.Mr/.cZqF7vYkRqKq8q8q8q8q8q8q8q

```
this.password = await bcrypt.hash(this.password, 10);  
  
}
```

```
// Lors de la connexion (comparaison sans pouvoir déchiffrer)  
  
async comparePassword(attempt: string): Promise<boolean> {  
  
return bcrypt.compare(attempt, this.password);  
  
}
```

Authentification JWT :

Après connexion réussie, le serveur génère un token JWT (JSON Web Token) que le client doit envoyer dans chaque requête sécurisée.

typescript

```
// Structure du token JWT (signé, non modifiable)  
  
{  
  
"sub": "550e8400-e29b-41d4-a716-446655440000", // id utilisateur  
  
"email": "admin@y-mad.mg",  
  
"role": "super_admin",  
  
"firstName": "Admin",  
  
"exp": 1735689600 // Expiration dans 7 jours  
  
}
```

Gestion des rôles et permissions :

Chaque route API peut être protégée par un ou plusieurs rôles autorisés.

typescript

```
// Seul un super_admin ou admin peut voir toutes les candidatures
```

```
@Get('applications/all')
```

```
@Roles(UserRole.SUPER_ADMIN, UserRole.ADMIN)
```

```
async getAllApplications() {
```

```
  return this.jobsService.getAllApplications();
```

```
}
```

```
// Un jeune connecté peut voir ses propres candidatures
```

```
@Get('applications/my')
```

```
@Roles(UserRole.MEMBER, UserRole.VOLUNTEER)
```

```
async getMyApplications(@CurrentUser() user: User) {
```

```
  return this.jobsService.getApplicationsByUser(user.id);
```

```
}
```

Protection contre les injections SQL :

L'utilisation de TypeORM (ORM) empêche les injections SQL car toutes les requêtes sont paramétrées.

typescript

```
// ❌ MAUVAIS : vulnérable aux injections SQL
```

```
const user = await query(`SELECT * FROM users WHERE email = '${email}'`);
```

```
// ❑ BON : TypeORM paramètre automatiquement
```

```
const user = await this.userRepository.findOne({ where: { email } });
```

Validation des données :

Toutes les données reçues du client sont validées avant traitement.

typescript

```
// DTO (Data Transfer Object) avec validations
```

```
export class ApplyToJobDto {
```

```
  @IsEmail({}, { message: 'Email invalide' })
```

```
  @IsNotEmpty()
```

```
  email: string;
```

```
  @IsString()
```

```
  @MinLength(3)
```

```
  fullName: string;
```

```
  @IsFile()
```

```
  @MaxFileSize(5 * 1024 * 1024) // 5 Mo
```

```
  @FileType(['pdf'])
```

```
  cv: File;
```

}

Journal d'audit (Audit Logs) :

Toute action importante est enregistrée dans la table audit_logs.

ChampExemple	Description
user_id550e8400-...	Qui a fait l'action
action "DELETE"	Type d'action
entity "job_application"	Table concernée
entity_id 123e4567-...	Identifiant de l'enregistrement
old_data {"status":"submitted"}	Avant modification
new_data {"status":"accepted"}	Après modification
ip_address "192.168.1.100"	Adresse IP de l'utilisateur
created_at 2025-06-15 14:30:00	Date et heure

4.3.4 Récapitulatif des Mesures de Sécurité

Menace	Mesure de protection	Mise en œuvre
Vol de mots de passe	Hachage bcrypt	Avant stockage en base
Interception des données	HTTPS/TLS 1.3	Certificat Let's Encrypt
Usurpation d'identité	JWT avec expiration	Token signé, valide 7 jours
Accès non autorisé	Guards par rôles	Vérification avant chaque route
Injection SQL ORM	TypeORM	Requêtes paramétrées
Attaque force brute	Rate limiting	100 requêtes/minute/IP
DDoS	Cloudflare	Filtrage au niveau CDN

Perte de données	Sauvegardes quotidiennes	Backup PostgreSQL automatique
Faible de sécurité	Journal d'audit	Toutes les actions tracées